

Análisis Numérico II / Álgebra Lineal Numérica

Clase 03: Terminando Cholesky + Eliminación Gaussiana



Facultad de Matemática,
Astronomía, Física y
Computación



Universidad
Nacional
de Córdoba

Algoritmo (Descomposición de Cholesky Prod Ext)

Entrada: Matriz "SDP" $A \in \mathbb{R}^{n \times n}$.

Salida: Matriz triangular superior $G \in \mathbb{R}^{n \times n}$ tal que $A = G^T G$.

Para $i = 1, \dots, n$ definir:

1. $g_{ii} = \sqrt{a_{ii}}$
2. Si $i < n$ entonces
 - $\mathcal{J} = \{i + 1, \dots, n\}$
 - $g_{i\mathcal{J}} = A_{i\mathcal{J}} / g_{ii}$
 - $A_{\mathcal{J}\mathcal{J}} \leftarrow A_{\mathcal{J}\mathcal{J}} - G_{i\mathcal{J}}^T G_{i\mathcal{J}}$
3. Retornar G .

¿Este algoritmo se puede romper? ¿Cómo?

Algoritmo (Descomposición de Cholesky)

Conteo Operacional

Siguiendo la lógica de la implementación por filas y columnas:

- **Actualización de la diagonal (G_{ii}):** Requiere un producto interno de vectores de longitud $i - 1$, una resta y una raíz cuadrada. El costo acumulado es de aproximadamente n^2 flops
- **Actualización fuera de la diagonal (G_{iJ}):** Es la parte más costosa debido a un bucle triplemente anidado que realiza productos y sumas para cada elemento. Este bloque suma aproximadamente $n^3/3$ flops

Conteo Total y Complejidad

El número exacto de operaciones para el algoritmo de producto interior es:

$$Costo = \frac{n^3}{3} + \frac{n^2}{2} + \frac{7n}{6}$$

Resultados Clave:

- La complejidad computacional dominante es $O(n^3/3)$.
- Las divisiones ($n^2/2$) y las raíces cuadradas (n) tienen un impacto insignificante en matrices de gran porte.

Cómo usamos todo lo aprendido para resolver un sistema lineal $Ax = b$?

1. El procedimiento requiere sustituir A por nuestra descomposición $G^T G$

$$Ax = b \iff G^T Gx = b$$

2. Luego agrupamos los factores

$$G^T(Gx) = b$$

3. Sea

$$y = Gx$$

4. Resolvemos $G^T y = b$

5. Resolvemos $Gx = y$

Algoritmo (Resolución de Sistema Lineal SDP)

Entrada: Sistema Lineal SDP $A \in \mathbb{R}^{n \times n}$ y vector $b \in \mathbb{R}^n$.

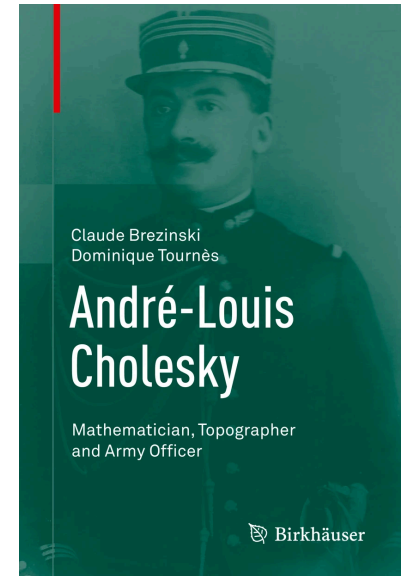
Salida: Vector solución $x \in \mathbb{R}^n$.

1. Calcular la descomposición Cholesky de A : $A = G^T G$.
2. Resolver el sistema triangular inferior $G^T y = b$ para y (sustitución hacia adelante).
3. Resolver el sistema triangular superior $Gx = y$ para x (sustitución hacia atrás).
4. Retornar x .

Cuántas operaciones estamos haciendo?

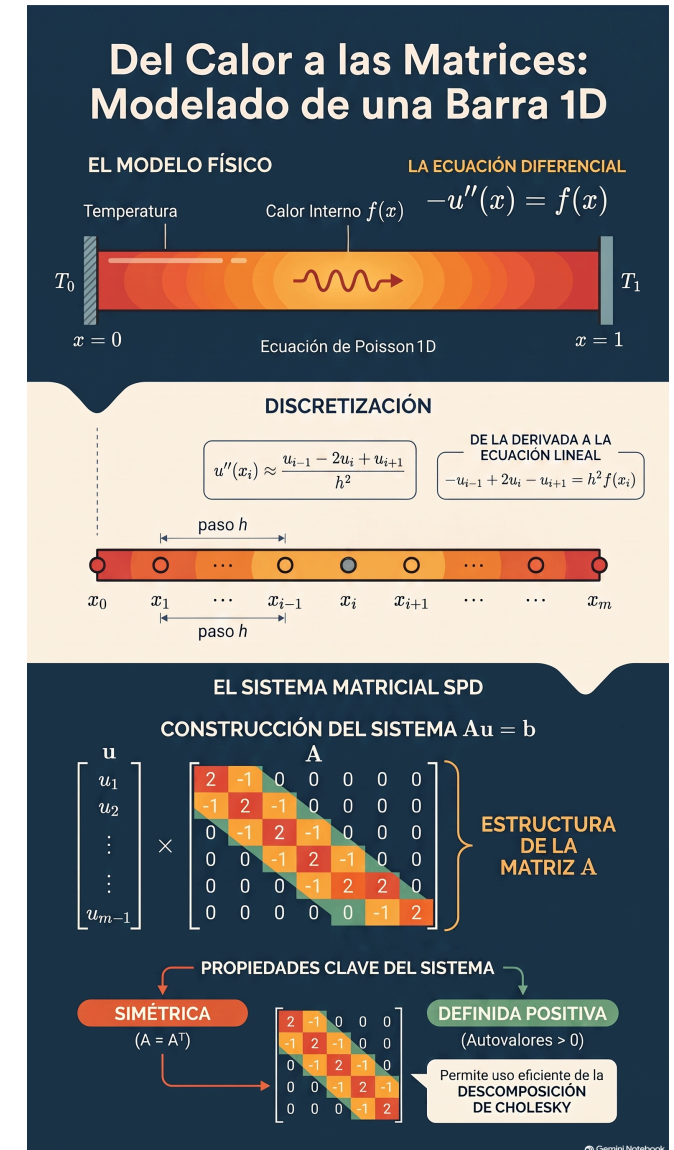
Un poco de Historia...

- Cholesky (1875–1918) era oficial de artillería e ingeniero geodesta del Service Géodésique de l'Armée Française.
- Propuesto en 1910 para resolver sistemas lineales provenientes de métodos de mínimos cuadrados en topografía.
- Cholesky no alcanzó a publicar su método, murió en la Primera Guerra Mundial :'(
- Ernest Benoit, compañero de Cholesky, lo publicó en 1924.



Aplicación y Costo Computacional

- Resolvemos sistemas lineales $Ax = b$ descomponiendo el problema en dos sistemas triangulares: $G^T y = b$ y $Gx = y$.
- Este método es fundamental en ingeniería y física, por ejemplo, para modelar la **distribución de temperatura** en una placa de acero.
- El costo operacional de la descomposición de Cholesky es de aproximadamente $O(n^3/3)$ flops.
- Spoiler: este algoritmo requiere aproximadamente la **mitad de operaciones** que una eliminación gaussiana.



Antes de Pasar a Una Nueva Descomposición

Además de ver un producto matricial por bloques, es útil verlo como "producto exterior" (outer product).

Si tengo dos matrices B y C , $A = BC$ puede verse como:

- $a_{ij} = \sum_{k=1}^n b_{ik}c_{kj}$, producto interno entre la fila i de B y la columna j de C

Pero también lo podemos ver como:

- $A = \sum_{k=1}^n B_{:,k}C_{k,:}$, donde $B_{:,k}$ es la columna k de B y $C_{k,:}$ es la fila k de C .

En vez de verla elemento por elemento, estamos viendo a A como una suma de matrices. *Qué rango tienen?*

Un Ejemplo Concreto

$$A = \underbrace{\begin{pmatrix} b_{11} \\ b_{21} \end{pmatrix}}_{B_{:,1}} \underbrace{\begin{pmatrix} c_{11} & c_{12} \end{pmatrix}}_{C_{1,:}} + \underbrace{\begin{pmatrix} b_{12} \\ b_{22} \end{pmatrix}}_{B_{:,2}} \underbrace{\begin{pmatrix} c_{21} & c_{22} \end{pmatrix}}_{C_{2,:}}$$

Primero, calculamos las dos matrices de rango uno:

$$\begin{pmatrix} b_{11}c_{11} & b_{11}c_{12} \\ b_{21}c_{11} & b_{21}c_{12} \end{pmatrix} + \begin{pmatrix} b_{12}c_{21} & b_{12}c_{22} \\ b_{22}c_{21} & b_{22}c_{22} \end{pmatrix}$$

Luego, las sumamos para obtener el resultado final:

$$A = \begin{pmatrix} b_{11}c_{11} + b_{12}c_{21} & b_{11}c_{12} + b_{12}c_{22} \\ b_{21}c_{11} + b_{22}c_{21} & b_{21}c_{12} + b_{22}c_{22} \end{pmatrix}$$

Por qué es importante esta perspectiva? 🤔

Aunque parezca más compleja, la perspectiva del producto exterior es crucial para el diseño de algoritmos.

- Muestra cómo se puede construir una matriz de forma **iterativa**.
- Muchos algoritmos de factorización (como vimos con Cholesky) se basan en la idea de "despegar" o restar estas componentes de rango uno de la matriz original, una a la vez, para revelar su estructura subyacente.
- Usaremos exactamente esta idea para deducir el próximo algoritmo.

Se acuerdan de Eliminación Gaussiana?

$$\text{Sea } A = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 10 \end{bmatrix}.$$

Multiplicamos f1 por -2 y se la restamos a f2, multiplicamos f1 por -3 y se la restamos a f3. Luego, multiplicamos f2 por -2 y se la restamos a f3:

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 10 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 4 & 7 \\ 0 & -3 & -6 \\ 0 & -6 & -11 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 4 & 7 \\ 0 & -3 & -6 \\ 0 & 0 & 1 \end{bmatrix}.$$

Transformaciones de Gauss

Si deseo transformar la columna k de A , en un vector con $x_i = 0$ para $i = k + 1, \dots, n$ sin alterar los elementos anteriores, defino

$$M_k = I - v^k (e^k)^T$$

donde $v^k = [0, \dots, 0, a_{k+1,k}/a_{kk}, \dots, a_{n,k}/a_{kk}]^T$ y e^k es el k -ésimo vector canónico.

Qué pinta tiene esta matriz? Podemos usarla para expresar la eliminación gaussiana?

La idea es realizar la premultiplicación de A por M_k para transformar la columna k de A en un vector con $a_{i,k} = 0$ para $i = k + 1, \dots, n$.

Volviendo al Ejemplo

$$\text{Sea } A = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 10 \end{bmatrix}.$$

"Multiplicamos f_1 por -2 y se la restamos a f_2 , multiplicamos f_1 por -3 y se la restamos a f_3 ". Luego, multiplicamos f_2 por -2 y se la restamos a f_3 .

$$\text{Es decir, premultiplicamos por } M_1 = I - \begin{bmatrix} 0 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -3 & 0 & 1 \end{bmatrix} \text{ y luego por}$$

$$M_2 = I - \begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix}$$

Eliminación Gaussiana

Para resolver $Ax = b$, aplicamos transformaciones de Gauss hasta obtener $Ux = y$ equivalente, donde

$$y = M_{n-1} \dots M_1 b, \quad U = M_{n-1} \dots M_1 A$$

luego U es un sistema triangular superior, entonces podemos resolver $Ux = y$ por **sustitución hacia adelante**.

Veamos el algoritmo de eliminación gaussiana de manera matricial, sin utilizar memoria extra.

Algoritmo (Eliminación Gaussiana)

Entrada: Sistema Lineal $A \in \mathbb{R}^{n \times n}$ y vector $b \in \mathbb{R}^n$.

Salida: Matriz triangular superior U y vector y tales que $Ux = y$.

1. Para $k = 1 \dots n - 1$, definir
 - $\mathcal{I} = \{k + 1, \dots, n\}$
 - $\mathcal{J} = \{k, \dots, n\}$
 - $v_{\mathcal{I}} = A_{\mathcal{I},k} / a_{k,k}$
 - $A_{\mathcal{I},\mathcal{J}} \leftarrow A_{\mathcal{I},\mathcal{J}} - v_{\mathcal{I}} A_{k,\mathcal{J}}$
 - $b_{\mathcal{I}} \leftarrow b_{\mathcal{I}} - v_{\mathcal{I}} b_k$
2. Retornar $U = A, y = b$.

Cuántas operaciones estamos haciendo?

Conteo Operacional

1. Para cada paso k tendremos $n - k$ productos para conseguir $v_{\mathcal{I}}$
2. $(n - k)(n - k + 1)$ productos y $(n - k)(n - k + 1)$ sumas para actualizar $A_{\mathcal{I},\mathcal{J}}$
3. $(n - k)$ productos y $(n - k)$ sumas para actualizar $b_{\mathcal{I}}$

Sumando todo tendremos aproximadamente $\frac{2}{3}n^3$ flops.

Cuánto tenemos si le sumamos la resolución del sistema lineal?

Descomposición LU

Tenemos un conjunto de Transformaciones de Gauss que aplicamos a izquierda para obtener una matriz triangular superior U .

$$U = M_{n-1} \dots M_1 A$$

Entonces

$$A = (M_{n-1} \dots M_1)^{-1} U = M_1^{-1} \dots M_{n-1}^{-1} U$$

Definimos $L = M_1^{-1} \dots M_{n-1}^{-1}$

Existen las inversas de las matrices M_k ?

PIZARRON

Teorema (Existencia y Unicidad de la Descomposición LU)

Sea $A \in \mathbb{R}^{n \times n}$. Si $\det(A_k) \neq 0$ para toda submatriz principal de tamaño $k \in \{1, \dots, n\}$, entonces existe una factorización única de A de la forma

$$A = LU$$

donde $L \in \mathbb{R}^{n \times n}$ es una matriz triangular inferior con $L_{ii} = 1$ para todo $i \in \{1, \dots, n\}$, y $U \in \mathbb{R}^{n \times n}$ es una matriz triangular superior.

Demostración (Existencia)

- Definimos $A^{(0)} = A$ y $A^{(k)} = M_k A^{(k-1)}$ para $k = 1 \dots n - 1$.
- Existencia: hemos construido las M_i , falta demostrar que están bien definidas, esto es: $(a_{k,k}^{(k-1)})$ es distinto de cero.
 - Caso base: $k = 1$, si $\det(A_{11}) \neq 0$ entonces $a_{11}^{(0)} \neq 0$.
 - Supongamos que existen transformaciones de Gauss tales que $A^{(k-1)} = \Gamma A$, con $\Gamma = M_{k-2} \dots M_1$.
 - Definimos $\mathcal{I} = \{1, \dots, k\}$ y $\mathcal{J} = \{k + 1, \dots, n\}$.

$$A^{(k)} = \Gamma A = \begin{bmatrix} \Gamma_{\mathcal{I},\mathcal{I}} & 0 \\ \Gamma_{\mathcal{J},\mathcal{I}} & \Gamma_{\mathcal{J},\mathcal{J}} \end{bmatrix} \begin{bmatrix} A_{\mathcal{I},\mathcal{I}} & A_{\mathcal{I},\mathcal{J}} \\ A_{\mathcal{J},\mathcal{I}} & A_{\mathcal{J},\mathcal{J}} \end{bmatrix}$$

Demostración (Existencia)

- Luego,

$$\det(A_{\mathcal{I},\mathcal{I}}^{(k-1)}) = \det(\Gamma_{\mathcal{I},\mathcal{I}} A_{\mathcal{I},\mathcal{I}}) = \det\left(\begin{bmatrix} a_{11}^{(k-1)} & \dots & a_{1k}^{(k-1)} \\ 0 & \ddots & \vdots \\ 0 & 0 & a_{kk}^{(k-1)} \end{bmatrix}\right) = \prod_{j=1}^k a_{jj}^{(k-1)}$$

- Como, por hipótesis, $\det(A_j) \neq 0$ para todo $j \in \{1, \dots, k\}$, entonces

$$a_{kk}^{(k-1)} \neq 0, \quad \forall k \in \{1, \dots, n\}$$

- Finalmente, todas las transformaciones de Gauss están bien definidas y puedo definir las matrices $L = (M_1 \dots M_{n-1})^{-1}$ y $U = A^{(n)}$.

Demostración (Unicidad)

- Supongamos que existen dos descomposiciones

$$A = L_1U_1 = L_2U_2$$

Algoritmo (Factorización LU)

Entrada: Matriz $A \in \mathbb{R}^{n \times n}$ con $\det(A_k) \neq 0$ para toda submatriz principal de tamaño $k \in \{1, \dots, n\}$.

Salida: Matrices $L \in \mathbb{R}^{n \times n}$ y $U \in \mathbb{R}^{n \times n}$ tales que $A = LU$.

1. Inicializar $L = I$ y $U = A$.
2. Para $k = 1 \dots n - 1$, definir
 - $\mathcal{I} = \{k + 1, \dots, n\}$
 - $\mathcal{J} = \{k, \dots, n\}$
 - $v_{\mathcal{I}} = U_{\mathcal{I},k} / u_{k,k}$
 - $L_{\mathcal{I},k} = v_{\mathcal{I}}$
 - $U_{\mathcal{I},\mathcal{J}} \leftarrow U_{\mathcal{I},\mathcal{J}} - v_{\mathcal{I}} U_{k,\mathcal{J}}$
3. Retornar L, U .